Esercizi Matlab

• Laboratorio 1

ES 1

Esercizio 1. Scrivere uno script `perditaprec.m` che stampa a schermo gli interi $a \in \{1, \ldots, 300\}$ per cui il risultato dell'operazione

```
1 - a*(1/a)
```

non è esattamente 0. Per verificare la corretta implementazione eseguirlo digitando `perditaprec` dal prompt e controllare che si ottenga il seguente output

```
soluzione(mia)
  for a = 1:300
      if 1 - a*(1/a) ~= 0
        fprintf('k = %d/n', a)
      end
  end
```

Inizializza a, tranne li zeri, fa il calcolo richiesto e li stampa.

A cosa serve questo esercizio?

Matematicamente $1 - a \cdot (1/a)$ **è sempre** zero, però nel computer i decimali non possono essere rappresentati con precisione, il computer utilizza un numero fisso di bit, per rappresentarli, 1/a viene arrotondato al numero intero più vicino rappresentabile, moltiplicando per a, l'errore si accumula.

Dunque non si vedranno TUTTI i numeri da 1 a 300. Vedo solo quelli che una volta trasformati in frazione causano un errore di precisione, (i numeri 1, 2, 8, 4 sono rappresentati perfettamente in binario)!

```
>> perditaprec
k = 49
k = 98
k = 103
k = 107
k = 161
k = 187
k = 196
k = 197
k = 206
k = 214
k = 237
k = 239
k = 249
k = 253
```

Es 2

Un esempio in cui la cancellazione numerica può essere (parzialmente) evitata è nel calcolo delle radici di un polinomio di secondo grado $ax^2 + bx + c$. La formula risolutiva è infatti

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a},$$

e contempla due potenziali sottrazioni. Quella sotto radice è inevitabile. Quella esterna può essere aggirata calcolando solo la soluzione in cui i segni dei due addendi sono concordi e ricavandosi l'altra sfruttando la relazione

$$x_1 \cdot x_2 = \frac{c}{a}.$$

Esercizio 2. Scrivere una funzione

```
function [x1, x2] = solve2(a,b,c)
```

che risolva l'equazione di secondo grado $ax^2 + bx + c = 0$ nel modo più stabile possibile, ovvero evitando la sottrazione più esterna. Testare l'implementazione su $x^2 - 10^{10}x +$

Il problema: cos'è la cancellazione numerica?

Quando si sottraggono due numeri **molto simili tra loro**, il computer perde cifre significative e il risultato diventa impreciso. Ad esempio:

> 1.000000001 − 1.000000000 = 0.000000001

Il computer può sbagliare le ultime cifre, e il risultato relativo diventa molto inaccurato.

Nella formula quadratica

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

ci sono **due sottrazioni potenzialmente pericolose:**

1. **Sotto la radice:** $b^2 - 4ac$ → inevitabile, non si può evitare
2. **Fuori dalla radice:** $-b \pm \sqrt{b^2 - 4ac}$ → qui se b è grande e positivo, il termine $-b + \sqrt{b^2 - 4ac}$ è una **sottrazione tra due nu**[illegible] **quasi uguali** → cancellazione!

dunque per evitare questo uso $x_1 \cdot x_2 = c/a$, se:

- $b >= 0$ $\quad x_1 = -b - \sqrt{\Delta} / 2a \quad x_2 = c/a \cdot x_1$
- $b < 0$ $\quad x_2 = -b + \sqrt{\Delta} / 2a \quad x_2 = c/a \cdot x_1$

} solo segni concordi!

soluzione mia

```
function [x1,x2] = solve2(a,b,c)

  delta = b^2 - 4a*c;
rad_delta = sqrt(delta)

  if b >=0
      x1 = (-b - rad_delta) / 2*a;
  else
      x1 = (-b + rad_delta) / 2*a;
  end
x2 = c/(a*x1)

end
```

```
>> function [x1, x2] = solve2(a, b, c)
       % Calcolo del discriminante
       delta = b^2 - 4*a*c;
       rad_delta = sqrt(delta);

       % Scegliamo il segno concorde a -b per evitare la sottrazione
       % Se b è positivo, usiamo il segno -
       % Se b è negativo, usiamo il segno +
       if b >= 0
           x1 = (-b - rad_delta) / (2*a);
       else
           x1 = (-b + rad_delta) / (2*a);
       end

       % Calcoliamo la seconda radice sfruttando x1 * x2 = c/a
       x2 = c / (a * x1);
   end

>> roots([10,4,3])

ans =

  -0.2000 + 0.5099i
  -0.2000 - 0.5099i
```

=

```
>> [r1,r2]=solve2(10,4,3)

r1 =

  -0.2000 - 0.5099i


r2 =

  -0.2000 + 0.5099i
```

ES 3

Esercizio 3. Scrivere una funzione `myexp3(x)` che controlla se x è negativo o positivo, e a seconda del segno calcola l'esponenziale come $1/e^{-x}$ oppure e^x con la serie di Taylor troncata a $n = 500$.

Per approssimare un'esponenziale, ci rifacciamo a Taylor

$$e^x = 1 + x + \underbrace{\frac{x^2}{2!}}_{a=a+x^k/k!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!} \quad \text{con } k$$

```
function a = myexp(x,n)
a= 1;
for k = 1:n
    a = a + x^k  / factorial(k);
end
end
```

2 moltiplicazioni

Però questa soluzione ogni volta fa $a = a + x^k / k!$ $O(n^2)$

Potrebbe invece fare $t = t \cdot x/1$ qua ne fa 1! è indipendente da x^k $O(n)$ più veloce

k	t (termine)	come si aggiorna t	a (somma)
inizio	t = 1	—	a = 1
k=1	t = 1 · 2/1 = **2**	t_precedente × x/k	a = 1 + 2 = **3**
k=2	t = 2 · 2/2 = **2**	t_precedente × x/k	a = 3 + 2 = **5**
k=3	t = 2 · 2/3 = **4/3**	t_precedente × x/k	a = 5 + 4/3 = **19/3**

Perché è furbo?

Nota che $x^2/2! = x \cdot x/2 = (x/1) \cdot (x/2)$. Ogni termine è il precedente **moltiplicato per x/k** — non serve ricalcolare tutto da zero, basta un'unica moltiplicazione per aggiornare t.

Una soluzione è la seguente

```
function a = myexp(x,n)
a= 1;
t = 1 ;
for k = 1:n
    t = t*x/k;
    a = a +t ;
end
end
```

Adesso ci sono problemi con i numeri negativi cioè se faccio e^{-30}, mi da un risultato negativo, ma deve essere positivo! con myexp(-30,500)

$$\underbrace{-30^{20}/20!}_{\text{positivo}} + \underbrace{-30^{21}/21!}_{\text{negativo}} = \text{si cancellano a vicenda! non va bene}$$

L'idea è di calcolare $e^{-30} = \frac{1}{30}$, e la soluzione è la seguente

```
function result = myexp3(x)
if x >= 0
    result = myexp(x,500);
end
if  x < 0
    result = 1/ myexp(-x,500);
end

end
```